

Detección de anomalías en facturas mediante Spark Streaming

Máster Universitario en Investigación en Inteligencia Artificial (UIMP)
Big Data – Práctica Spark Streaming

Descripción

Este proyecto implementa una pipeline de procesamiento en tiempo real utilizando **Apache Spark Streaming** y **Apache Kafka** sobre el conjunto de datos **Online Retail**.

La aplicación consume compras individuales desde Kafka, reconstruye las facturas a partir de sus líneas, extrae las características de cada factura y detecta posibles anomalías utilizando dos modelos de clustering ejecutados en paralelo:

- KMeans
- Bisecting KMeans

Además, durante el procesamiento también:

- identifica compras con información incompleta o incorrecta;
 - contabiliza las facturas canceladas en una ventana deslizante de 8 minutos;
 - publica todos los resultados en distintos topics de Kafka.
-

Estructura del proyecto

```
.
├── build.sbt
├── env.sh
├── README.md
├── start_training.sh
├── start_pipeline.sh
├── productiondata.sh
├── resources
│   ├── production.csv
│   └── training.csv
├── project/
├── src/
│   ├── main/
│   │   ├── scala/
│   │   └── es/
```

```
dmr/  
  uimp/  
    clustering/  
    realtime/  
    simulation/
```

También durante la ejecución, el proyecto genera los siguientes archivos (por limpieza los incluimos en `.gitignore`):

```
target/  
checkpoint/  
spark-warehouse/  
clustering/  
clustering_bisect/  
threshold  
threshold_bisect
```

Requisitos

El proyecto se ha desarrollado utilizando:

- Java 8
- Scala 2.11
- Apache Spark 2.3.1
- Apache Kafka 0.8.2.1
- sbt

Antes de compilar o ejecutar es necesario cargar las variables de entorno:

```
source ./env.sh
```

Compilación

Para generar el archivo ejecutable:

```
source ./env.sh  
sbt clean assembly
```

Al finalizar genera:

```
target/scala-2.11/anomalyDetection-assembly-1.0.jar
```

Entrenamiento de los modelos

Los modelos de detección de anomalías se entrenan ejecutando:

```
./start_training.sh
```

Este script:

- entrena el modelo KMeans;
- entrena el modelo Bisecting KMeans;
- guarda ambos modelos entrenados;
- calcula y almacena los umbrales de detección.

Al finalizar aparecerán los siguientes elementos:

```
clustering/  
threshold
```

```
clustering_bisect/  
threshold_bisect
```

Pueden explorarse los umbrales de detección de anomalías:

```
cat threshold  
cat threshold_bisect
```

Ejecución de la pipeline

Una vez entrenados los modelos, la aplicación puede ejecutarse siguiendo el siguiente orden.

1. Iniciar Zookeeper

```
$KAFKA_HOME/bin/zookeeper-server-start.sh \  
$KAFKA_HOME/config/zookeeper.properties
```

2. Iniciar Kafka Broker

```
$KAFKA_HOME/bin/kafka-server-start.sh \  
$KAFKA_HOME/config/server.properties
```

3. Iniciar la pipeline de Spark Streaming

```
./start_pipeline.sh
```

4. Lanzar el productor de datos

```
./productiondata.sh
```

El orden de ejecución es importante:

1. Zookeeper
 2. Kafka Broker
 3. Spark Streaming
 4. Productor de datos
-

Topics de Kafka

Entrada

purchases

Salida

facturas_erroneas
cancelaciones
anomalias_kmeans
anomalias_bisect_kmeans

Arquitectura de la solución

production.csv

InvoiceDataProducer

Kafka (purchases)

InvoicePipeline

Facturas erróneas
Cancelaciones
Reconstrucción de facturas
Detección mediante KMeans
Detección mediante Bisecting KMeans

Topics de salida de Kafka

Comprobación del funcionamiento

Una vez iniciada la aplicación, el contenido de los topics puede consultarse utilizando el consumidor de Kafka.

Por ejemplo:

```
source ./env.sh
```

```
$KAFKA_HOME/bin/kafka-console-consumer.sh \
  --zookeeper localhost:2181 \
  --topic anomalias_kmeans \
  --from-beginning
```

De forma equivalente pueden consultarse:

- facturas_erroneas
- cancelaciones
- anomalias_bisect_kmeans

Durante la ejecución también pueden observarse en la consola de Spark los contadores de:

- compras válidas;
 - compras inválidas;
 - facturas reconstruidas;
 - cancelaciones;
 - anomalías detectadas mediante ambos modelos.
-

Ejecución mediante alias (opcional)

Durante el desarrollo se utilizaron varios alias de Bash para facilitar la puesta en marcha del sistema.

Añadiendo las siguientes líneas al archivo ~/.bashrc es posible lanzar cada componente en una terminal independiente.

```
export BD_PROJECT_HOME="/ruta/al/proyecto/codigo-sparkinvoice"
```

```
alias bd-zk='gnome-terminal --title="01 - Zookeeper" -- bash -lc "cd \"$BD_PROJECT_HOME\" &&
```

```
alias bd-kafka='gnome-terminal --title="02 - Kafka Broker" -- bash -lc "cd \"$BD_PROJECT_HOM
```

```
alias bd-pipeline='gnome-terminal --title="03 - Spark Pipeline" -- bash -lc "cd \"${BD_PROJE'
```

```
alias bd-producer='gnome-terminal --title="04 - Producer" -- bash -lc "cd \"${BD_PROJECT_HOM'
```

Después de modificar el archivo:

```
source ~/.bashrc
```

Cada temrinal puede lanzarse individualmente:

```
bd-zk
bd-kafka
bd-pipeline
bd-producer
```

En nuestro caso diseñamos la siguiente función que lanza todas las terminales:

```
bd-run() {
    bd-zk
    sleep 5

    bd-kafka
    sleep 8

    bd-pipeline
    sleep 10

    bd-producer
}
```

De tal forma que podemos tener el sistema de 4 elementos corriendo con un solo comando:

```
bd-run
```

Esta configuración es completamente opcional y únicamente pretende facilitar la ejecución local del proyecto, que en nuestro caso ha sido de gran utilidad.

Notas

- Los modelos entrenados se generan automáticamente mediante `start_training.sh`.
- Los modelos, checkpoints y demás archivos temporales no se incluyen en el repositorio y pueden regenerarse en cualquier momento.
- Se ha mantenido la estructura original del proyecto proporcionada en el enunciado, simplificando y comentando en la medida de lo posible el código y los scripts para facilitar su comprensión y reproducción.